

Model	1 Epoch	2 Epochs
XL-ORIGINAL	1.926%	1.571%
XL-NEARDUP	0.189%	0.264%
XL-EXACTSUBSTR	0.138%	0.168%

Table 4: When generating 100k sequences with no prompting, over 1% of the tokens emitted from a model trained on the original dataset are part of a 50-token long sequence copied directly from the training dataset. This drops to 0.1% for the deduplicated datasets.

much larger than the variation of about 1 point perplexity we observed in the base models. This is despite seeing fewer tokens of training data overall.

Lastly, we note all our XL models achieved <35 perplexity on LM1B, which is less than the 42.16 perplexity reported for the 1.5B GPT-2 using a vocabulary the same size as ours.

6.2 Generated Text

Data duplication has the effect of biasing the trained LM towards particular types of examples. This can contribute to a lower diversity of generations, and increased likelihood that the generated content is copied from the training data (Carlini et al., 2020). For our generation experiments, we use top- k random sampling with $k = 50$ and experiment with prompted and unprompted generation.

No prompt. We first evaluate memorization tendencies in the case where the model is asked to generate text without any prompt sequence. We generate 100,000 samples, each up to 512 tokens in length (examples provided in the Appendix). For each generated token, we say the token is memorized if it is part of a 50-token substring that is exactly contained in the training data. On XL-ORIGINAL, over 1% of the generated tokens belong to memorized sub-sequences (see Table 4). This is $\sim 10\times$ more memorization than XL-EXACTSUBSTR or XL-NEARDUP. Some example subsequences that were copied verbatim from the train set can be found in Table 9 in the Appendix.

With prompting. In most real use cases, language model generation is controlled by providing a prompt for the model to continue. We experiment with four possible prompt sources: training examples identified by EXACTSUBSTR as having near-duplicates in the train set (train dup), training examples identified as unique (train unique), validation set examples with a near-duplicate in the train set (valid in train), and validation set ex-

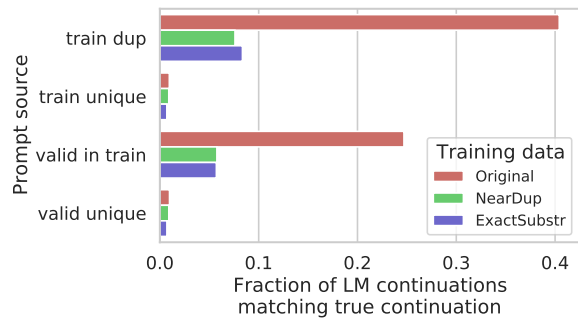


Figure 3: The proportion of generations which have edit similarity above 0.8 with the groundtruth continuation when using the LM to generate continuations for 32-token prompts identified by NEARDUP as either duplicated or unique.

Model	Dataset	Orig	Dups	Unique
Transformer-XL	LM1B	21.77	10.11	23.58
GROVER-Base	RealNews	15.44	13.77	15.73
GROVER-XL	RealNews	9.15	7.68	9.45

Table 5: For each model, the perplexity of the official validation set (*Orig*), valid set examples which were identified by NEARDUP as matches of train set examples (*Dups*), and valid set examples identified by NEARDUP as unique (*Unique*). Due to the size of the RealNews validation set, we evaluated on only the first 25k examples meeting each condition.

amples identified as unique across all splits (valid unique). We select the first 32 tokens of each example as the prompt, which means we can evaluate the fraction of generations which are near-duplicates with the ground-truth continuation for the prompt (Figure 3). When the prompt comes from duplicate examples in the train set, XL-ORIGINAL reproduces the groundtruth continuation over 40% of the time. XL-EXACTSUBSTR and XL-NEARDUP still copy the groundtruth more often when the prompt comes from a duplicate example than when the prompt comes from a unique example, suggesting that more stringent deduplication may be necessary to remove memorization tendencies entirely.

6.3 Impact on Existing Models

Train-test leakage does not just impact models trained on C4. Table 5 shows that the presence of near-duplicates of the evaluation set in the train set has a significant impact on model perplexity for two standard models: Transformer-XL (Dai et al., 2019), which was trained on LM1B, and GROVER (Zellers et al., 2019), which was trained on RealNews. For Transformer XL, the perplexity

halves on examples identified as near-duplicates. For GROVER, the difference, though not quite as stark, is present in both model sizes considered.

Existing models also suffer from the problem of generating text from their train sets. We find that 1.38% of the tokens in the official release of 25k GROVER-Mega outputs³ are part of verbatim matches in RealNews of at least length 50. Likewise, more than 5% of the tokens in ~200k sequences outputted by GPT-Neo 1.3B (Black et al., 2021) are part of a 50 token matches of its training data, the Pile (Gao et al., 2020).

7 Discussion

The focus of this paper is on the datasets used to train language models. While recent work focused on documenting the potential harms that could arise from problematic datasets (Bender and Friedman, 2018; Gebru et al., 2020), less work has been done to quantitatively analyze properties of real language modelling datasets, like Dodge et al. (2021a) has done for C4. Our paper provides analysis on one particular axis, that of data duplication.

Our experiments measured what could be quantified: the amount of duplicate content in common datasets, the effect of deduplication on trained model perplexity, and the reduction of memorized content in trained models through deduplication. We do not focus on the nature of the data being removed by deduplication or memorized by LMs.

Privacy is an important subject for future work, as memorized training data has significant privacy consequences. By this, we mean the standard privacy definition that a model should not reveal anything particular to the specific dataset it was trained on, as opposed to another training dataset from a similar distribution (Shokri et al., 2017).⁴ Training on standard datasets that have not yet been deduplicated results in models that are particularly sensitive to examples that happened to be repeated multiple times, and this has negative privacy implications. For instance, it could violate a person’s expectations of privacy if their publicly available personal data appeared in a different, surprising context. Downstream applications of LMs, such

as the game AI Dungeon⁵, should also not output memorized content like adverts for real products.

We stress that in our experiments, we do not distinguish between undesired memorized text (such as phone numbers), innocuous memorized text (common phrases), and text we may want to be memorized (such as a quote by a public figure), and instead treat all instances of the LM generating text that closely matches the training set as problematic. While we qualitatively observed that much of the identified memorized content was relatively innocuous, a more systematic study of the risks associated with the detected memorization was beyond the scope of this work.

We also do not investigate the negative consequences of deduplication. Some language tasks explicitly require memorization, like document retrieval or closed-book question answering. Also, text that gives attribution is often duplicated across documents, so removing duplicate substrings could correspond to removing *just* the attribution, which could result in models that learn the content without its attached attribution. Deduplication is also not sufficient to remove privacy-sensitive data like bank passwords and medical records which should never be used in training data (Brown et al., 2022).

Ultimately, whether memorization is a desired property of a language model, or else risky and unwanted, depends both on the nature of the text that has been memorized and on the downstream applications of the trained model. However, since the trend has been towards creating datasets and models that are application-agnostic, we encourage researchers to think carefully about the limitations of the data they have collected and the how the model’s intended usage constrains what should be part of the training set. Developing techniques to memorize or forget specific sequences depending on the end application is a promising research direction.

8 Conclusion

We encourage future language model research to perform dataset deduplication, either by training on the deduplicated datasets we release, using the deduplication tools we release, or following our approach to deduplicate datasets with new tools.

The exact technique used to perform deduplication is less important than performing stringent deduplication in the first place. On the whole, dedu-

³[gs://grover-models/generation_examples/generator=mega-dataset=p0.90.jsonl](https://grover-models/generation_examples/generator=mega-dataset=p0.90.jsonl)

⁴Another interpretation of privacy focuses on the sensitivity of the data involved, when a model is trained on and able to reproduce personal identifiers or other forms of “private data.” Our definition is more expansive.

⁵<https://play.aidungeon.io/>

plication does not harm, and sometimes improves, model perplexity, despite the fact that the deduplicated datasets are smaller and faster to train on. It is especially important that there are no duplicates between the training and testing sets, because overlap here explicitly encourages selecting models that memorize the training data. Lastly, deduplication helps to reduce some of the privacy concerns around LMs memorizing their training data.

Ethics

The developers of large language models typically attempt to create training data that reflects natural human communication, but current methods to collect and curate such datasets are fallible. There are multiple reasons some text ends up over-represented. For example, bot replies, auto-generated templates, and licenses are repeated for structural (e.g., legal, economical) reasons (as was also observed by [Dodge et al. \(2021a\)](#)). Additionally, common techniques for acquiring and “cleaning” data can result in an over-representation of particular subsets of world users, often those who are English-speaking and publishing in established forums. This effectively under-represents non-English speakers as well as groups whose communication mostly occurs outside of the public web. In this paper, we focus on the problem of over-representation of some types of text (structural duplicates) but do not address the problem of under-representation of others.

Additionally, while we discuss when memorized content might be desired and when it might not be desired, our analysis does not disambiguate these two cases. Work to disambiguate helpful from harmful memorization is tremendously complex and would require a different set of research methodologies than are presented in this work.

Acknowledgements

We are grateful to the many researchers whose technical help, feedback, and discussions shaped this project: Jacob Austin, Samy Bengio, Olivier Bousquet, James Bradbury, Fernando Diaz, Mark Diaz, Noah Fiedel, Jonathan Frankle, David Grangier, Stefanie Karp, David Mimno, Gaurav Mishra, Michael Mozer, Sharan Narang, Alex Pasos, Adam Roberts, Hanie Sedghi, Jascha Sohl-dickstein, David So, Florian Tramèr, and Yun William Yu. We are also grateful to the Google

Brain women who have given us continuous support.

Chris Callison-Burch and Daphne Ippolito’s research is supported in part by the DARPA KAIROS Program (contract FA8750-19-2-1004), the DARPA LwLL Program (contract FA8750-19-2-0201), and the IARPA BETTER Program (contract 2019-19051600004). The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of DARPA, IARPA, or the U.S. Government.

Contributions

Each of the authors on this paper significantly contributed to the final results.

- Katherine trained the models used in the paper, built and ran the eval and text generation pipelines, contributed significantly to writing, analysis, and project organization and management.
- Daphne ran the approximate matching data deduplication pipelines, extracted prompts and evaluation datasets, ran eval pipelines, and contributed significantly to planning, writing, and analysis.
- Andrew wrote the code to perform deduplication with approximate matching, helped evaluate energy expenditure, and helped with analysis.
- Chiyan helped generate plots and contributed to project scoping, writing, and data analysis.
- Chris offered mentorship and guidance throughout the project and contributed to writing.
- Doug offered mentorship and guidance throughout the project and contributed to writing.
- Nicholas wrote the suffix array implementation, ran all EXACTSUBSTR deduplication experiments, contributed significantly to planning, writing, and analysis, as well as scoping the project.

References

- Miltiadis Allamanis. 2019. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 143–153.